

update_engine_integration

Helix OTA — update_engine Integration (1.0.0-MVP)

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	The device-side apply contract for the Helix OTA Android agent: UpdateEngine.applyPayload(url, offset, size, props) with the four header properties (FILE_HASH / FILE_SIZE / METADATA_HASH / METADATA_SIZE), the UpdateEngineCallback status constants + error codes, the rationale for the local-verified-file apply path, and the AVB / boot_control / update_verifier interplay that delivers the no-corruption guarantee. Constants are carried verbatim from the verified AOSP research notes; UNVERIFIED items are flagged. Implemented behind the ota-update-engine-bridge (NEW, Android-only) submodule.
Issues	CLEANUP_PREVIOUS_UPDATE status value, the complete Android-15 ErrorCodeConstants table (codes 13..50), and whether to bind via IUpdateEngine vs the stable AIDL surface are UNVERIFIED. Target-board AVB lock state, boot_control HAL conformance, and rollback-index storage on RK3588 / Orange Pi 5 Max are UNVERIFIED and require hardware validation. HelixConstitution clause numbers are UNVERIFIED.

Field	Value
Fixed	N/A (initial revision).
Continuation	Pin the Android-15 ErrorCodeConstants table and CLEANUP_PREVIOUS_UPDATE value against android-15.0.0_rNN; decide IUpdateEngine vs IUpdateEngineStable; confirm SELinux policy for the Helix-signed system app to reach update_engine; validate AVB lock + boot_control + rollback-index backend on the target board.

Table of contents

1. [Purpose and scope](#)
2. [The applyPayload contract](#)
3. [The four header properties](#)
4. [UpdateEngineCallback: status constants](#)
5. [UpdateEngineCallback: error codes](#)
6. [Local-verified-file apply rationale](#)
7. [AVB / boot_control / update_verifier interplay](#)
8. [What the agent MUST NOT do](#)
9. [The bridge surface \(ota-update-engine-bridge\)](#)
10. [Testing \(four-layer\)](#)
11. [Open / UNVERIFIED items](#)
12. [Sources](#)

ToC mandated by HelixConstitution §11.4.61 (UNVERIFIED clause number).

1. Purpose and scope

This spec is the **device apply contract** the Helix Android agent implements via the **ota-update-engine-bridge** (NEW, Android-only, thin, testable; [submodule reuse map §4](#)). It pins exactly how the agent hands a verified artifact to AOSP update_engine, what callbacks/codes it must handle, and how that sits on top of the AVB / boot_control / update_verifier safety stack.

It honors the LOCKED strategy: **native Android A/B** (update_engine + AVB/dm-verity + auto-rollback) on device [ADR-0001 §1]; **signing + SHA-256 + AVB** for MVP, TUF device-side deferred to 1.0.1 [ADR-0002 §4.1, §4.3]. The agent **wraps**, **never replaces** the engine [ADR-0001; android-avb-rollback §10].

All constants below are carried from the verified research notes [android-update-engine-api](#); where a value could not be pinned to a single Android-15 source it is marked UNVERIFIED.

2. The applyPayload contract

android.os.UpdateEngine is @SystemApi. Verified overloads [android-update-engine-api §5]:

```
// Streaming or local-by-URL form
public void applyPayload(String url, long offset, long size,
                        String[] headerKeyValuePairs)

// File-descriptor form (AssetFileDescriptor variant)
public void applyPayload(@NonNull AssetFileDescriptor assetFd,
                        @NonNull String[] headerKeyValuePairs)
```

Callback registration:

```
public boolean bind(final UpdateEngineCallback callback)
public boolean bind(final UpdateEngineCallback callback, final
                    Handler handler)
```

Call flow the bridge implements [android-update-engine-api §5]:

1. UpdateEngine engine = new UpdateEngine();
2. engine.bind(callback);
3. Read the four lines from payload_properties.txt into String[] props.
4. Compute offset/size of the **payload.bin entry within the outer zip** (not the whole zip).
5. engine.applyPayload(url, offset, size, props);
 - url = "file:///data/ota_package/ota_update.zip" for local apply (MVP default), OR
 - url = "https://.../ota_update.zip" for streaming (opt-in).
6. Drive telemetry from onStatusUpdate; finalize on onPayloadApplicationComplete(SUCCESS), then reboot.

offset/size semantics: for both file:// and https://, offset/size describe the **payload.bin entry's byte position inside the outer zip**; update_engine reads only that range as the payload [android-update-engine-api §5]. For the streaming form, every zip entry must be stored **uncompressed (ZIP_STORED)** and the host must support HTTP **Range** [android-update-engine-api §9].

Kotlin: see [code_snippets/UpdateApplier.kt](#) and [code_snippets/UpdateEngineCallbackImpl.kt](#).

3. The four header properties

payload_properties.txt (generated by brillo_update_payload) contains exactly four KEY=VALUE lines, passed **verbatim** as headerKeyValuePairs [android-update-engine-api §4, §6]:

Property	Type	Meaning	Mismatch → error
FILE_HASH	base64(SHA-256)	Hash of the entire payload.bin	PAYLOAD_HASH_MISMATCH_ERROR (10)

Property	Type	Meaning	Mismatch → error
FILE_SIZE	bytes (decimal)	Total length of payload.bin	PAYLOAD_SIZE_MISMATCH_ERROR (11)
METADATA_HASH	base64(SHA-256)	Hash of the metadata prefix (first METADATA_SIZE bytes); verified early	metadata-signature errors (24/25/26)
METADATA_SIZE	bytes (decimal)	Length of the metadata prefix region	—

Example values [android-update-engine-api §4]:

```
FILE_HASH=lURPCIkIAjtM0yB/EjQcl8zDzqtD6Ta3tJef6G/+z2k=
FILE_SIZE=871903868
METADATA_HASH=tBvj43Q0B0Jn++JojcpVdbRLz0qdAuL+uTkSy7hokaw=
METADATA_SIZE=70604
```

METADATA_HASH/METADATA_SIZE let the engine validate the small leading metadata region **before** committing to the full download — fail-fast, important for the streaming path [android-update-engine-api §6]. These four values must flow from the Helix build pipeline into the control-plane release manifest so the agent can both pass them to `applyPayload` and pre-verify FILE_HASH/FILE_SIZE (see [integration_guide.md §8](#)).

4. UpdateEngineCallback: status constants

`UpdateEngine.UpdateStatusConstants` (verified integer values) [android-update-engine-api §8]:

Constant	Value
IDLE	0
CHECKING_FOR_UPDATE	1
UPDATE_AVAILABLE	2
DOWNLOADING	3
VERIFYING	4
FINALIZING	5
UPDATED_NEED_REBOOT	6
REPORTING_ERROR_EVENT	7
ATTEMPTING_ROLLBACK	8
DISABLED	9
CLEANUP_PREVIOUS_UPDATE	UNVERIFIED — present on Virtual A/B trees; commonly 12 in <code>update_status.h</code> ; confirm for Android 15

onStatusUpdate(int status, float percent) is the progress signal; the bridge maps each status into a Helix telemetry state ([integration_guide.md §9](#)). UPDATED_NEED_REBOOT (6) is the cue that the payload is applied and a reboot will switch slots.

5. UpdateEngineCallback: error codes

UpdateEngine.ErrorCodeConstants (verified where literal-sourced) [android-update-engine-api §8]:

Constant	Value	Notes
SUCCESS	0	apply succeeded
ERROR	1	generic error
FILESYSTEM_COPIER_ERROR	4	
POST_INSTALL_RUNNER_ERROR	5	post-install script failed
PAYLOAD_MISMATCHED_TYPE_ERROR	6	full vs incremental mismatch
INSTALL_DEVICE_OPEN_ERROR	7	
KERNEL_DEVICE_OPEN_ERROR	8	
DOWNLOAD_TRANSFER_ERROR	9	network/transfer failure (streaming)
PAYLOAD_HASH_MISMATCH_ERROR	10	FILE_HASH mismatch
PAYLOAD_SIZE_MISMATCH_ERROR	11	FILE_SIZE mismatch
DOWNLOAD_PAYLOAD_VERIFICATION_ERROR	12	payload signature/verification failed
PAYLOAD_TIMESTAMP_ERROR	51	target build older than current (anti-rollback)
UPDATED_BUT_NOT_ACTIVE	52	applied but slot not made active
NOT_ENOUGH_SPACE	60	insufficient space (Virtual A/B COW)
DEVICE_CORRUPTED	61	

Underlying error_code.h values relevant to verification failures [android-update-engine-api §8]: kDownloadPayloadPubKeyVerificationError=18, kDownloadMetadataSignatureError=24, kDownloadMetadataSignatureVerificationError=25, kDownloadMetadataSignatureMismatch=26,

kDownloadOperationHashVerificationError=27. Codes 13..50 surface inconsistently between the C++ enum and Java constants — UNVERIFIED for the exact Android-15 tag.

Agent handling: map every non-SUCCESS code into a telemetry error event and abort cleanly. PAYLOAD_TIMESTAMP_ERROR (51) means the control plane attempted an anti-rollback-violating downgrade; the engine refuses it [android-update-engine-api §12] — the rollout policy must never push an older build to a newer device. NOT_ENOUGH_SPACE (60) correlates with low free /data for the Virtual A/B COW [android15-virtual-ab §8] and should be reported with the device's free-/data reading.

6. Local-verified-file apply rationale

Both apply modes use the same applyPayload; only the url scheme differs. The MVP default is **local file://** [android-update-engine-api §7; ADR-0002 §4.1]:

Aspect	Local (file://) — MVP default	Streaming (https://) — opt-in
Who downloads	The Helix agent downloads the full zip to /data/ota_package/, then points the engine at it	update_engine downloads via HTTP Range as it applies
Disk usage	Needs room for the whole package on /data	Minimal extra storage
Verify-before-apply	Strong — agent verifies whole-zip signature and FILE_HASH/METADATA_HASH before calling applyPayload	Weaker — engine verifies during/after; a poisoned stream is only caught mid-apply
Network during apply	None (already on disk)	Live connection for the whole apply; CDN must support Range + ZIP_STORED
Resumability	Agent controls retry/resume	Engine resumes the transfer

The local path gives **two independent verification layers**: (1) the agent verifies SHA-256 + signature + the four hashes before apply, then (2) update_engine re-verifies metadata + payload signature — defense-in-depth consistent with the Uptane-ready trust direction [android-update-engine-api §7; ADR-0002 §4.1]. This is why ADR-0002 §4.1 attributes the local-verified-file decision to the **verify-before-apply** driver. Streaming remains available for storage-constrained fleets but forfeits the pre-apply full-artifact verification window.

7. AVB / boot_control / update_verifier interplay

The apply is only the entry point; the **no-corruption guarantee** comes from the on-device, signed, bootloader-enforced stack underneath [android-avb-rollback §9; android15-virtual-ab §9]:

1. **A/B slots (update_engine)**. applyPayload writes only the **inactive** slot (Virtual A/B: into the COW snapshot via snapuserd); the running slot is never touched [android15-virtual-ab §9].
2. **AVB / vbmeta**. The image is cryptographically authenticated to the OEM (or user) root key; the bootloader reports androidboot.verifiedbootstate (GREEN/YELLOW/ORANGE/RED). Production fleets should be **GREEN** (or YELLOW for a custom-key fleet) [android-avb-rollback §3].
3. **boot_control HAL**. After writing, update_engine calls setActiveBootSlot(slot) (clears unbootable/successful, resets retry count). On the next boot the device enters the new slot in a **trial state** (bootable, not-yet-successful, retry-count > 0) [android-avb-rollback §5, §7].
4. **update_verifier**. On first boot into the new slot it reads the **care map** and forces reads so **dm-verity** verifies the freshly written blocks before commit. Only on success does the path to **markBootSuccessful** proceed [android-avb-rollback §6].
5. **Automatic rollback**. If the slot is never marked successful and slot-retry-count hits zero, the bootloader falls back to the other (known-good) slot — bootloader-enforced, no network/agent needed [android-avb-rollback §7].
6. **Anti-downgrade (rollback index)**. Boot is permitted only if image.rollback_index >= stored_rollback_index; the load-bearing ordering rule is **mark SUCCESSFUL before bumping the stored index**, else a power loss can leave no bootable slot [android-avb-rollback §8].
7. **Virtual A/B merge**. After a confirmed-good boot, snapuserd merges the COW into the base; merge is power-fail-resumable. The **dangerous window** is reboot-to-old-slot **after merge has started** — forbidden [android15-virtual-ab §9, §10].

The agent **observes** this stack for telemetry (ro.boot.verifiedbootstate, ro.boot.veritymode, current/active slot, slot-successful, MergeStatus) but **drives** it only through documented APIs [android-avb-rollback §10]. See [code_snippets/BootControlObserver.kt](#).

8. What the agent MUST NOT do

From [android-avb-rollback §10](#), the agent **MUST NOT**:

- write slot flags (active/bootable/successful) or alter slot-retry-count outside update_engine / documented HAL usage;
- call markBootSuccessful itself to force a commit (defeats update_verifier and the trial window);
- write or bump the **rollback index**, or reorder the “mark SUCCESSFUL before bump index” sequence;
- regenerate, strip, or re-sign **vbmeta** on-device, disable AVB, or boot with verification off;
- disable dm-verity or mount protected partitions read-write to patch in place;
- assume the agent must be alive for rollback (rollback is bootloader-enforced);

- trigger a slot revert once MergeStatus == MERGING [android15-virtual-ab §10].

9. The bridge surface (ota-update-engine-bridge)

The bridge is the **only** OS-apply path [submodule reuse map §4]. Its surface is intentionally minimal:

- applyVerifiedPackage(file: LocalArtifact, props: PayloadProperties): ApplyHandle — binds the callback, computes offset/size for the payload.bin entry, and calls applyPayload(file://..., offset, size, props).
- observeStatus(): Flow<EngineStatus> — maps onStatusUpdate/onPayloadApplicationComplete to typed events.
- currentSlot(), verifiedBootState(), mergeStatus() — read-only observers for telemetry.
- No polling, no networking, no business policy in the bridge.

The bridge accepts only an **already-verified** local artifact; verification lives in the agent (Security-KMP), not the bridge, keeping the bridge thin and the safety gate centralized.

10. Testing (four-layer)

Per HelixConstitution §1 / §1.1 and master design §13; apply is safety-critical, floored at **≥90%** [master design §13; additions_synthesis §5 C8]:

1. **Source-presence gate.** Assert applyPayload is called with the four props in order; the bridge exposes no slot-flag / markBootSuccessful / rollback-index writer (grep gate proving the MUST-NOTs of §8 are absent from source).
2. **Artifact gate.** The shipped bridge bytes contain the callback handler and the applyPayload call; no forbidden HAL mutators are present in the compiled module.
3. **Runtime / integration.** Host/emulated update_engine fake exercises applyVerifiedPackage → onStatusUpdate(3..6) → onPayloadApplicationComplete(SUCCESS) over a real ZIP_STORED payload; inject PAYLOAD_HASH_MISMATCH_ERROR (10) and PAYLOAD_TIMESTAMP_ERROR (51) and assert correct telemetry + abort. Real-board plan: download → verify → apply → reboot → verify; corrupt-slot → confirm automatic A/B fallback [master design §13].
4. **Mutation meta-test.** Negate a safety invariant (e.g. make the bridge call markBootSuccessful, or pass props out of order, or revert after MERGING) and assert tests flip PASS → FAIL.

11. Open / UNVERIFIED items

- CLEANUP_PREVIOUS_UPDATE value; complete Android-15 ErrorCodeConstants (13..50) — UNVERIFIED [android-update-engine-api §13].

- IUpdateEngine vs stable IUpdateEngineStable binding choice — UNVERIFIED [android-update-engine-api §13].
- SELinux policy for the Helix-signed system app to reach update_engine — UNVERIFIED.
- RK3588 / Orange Pi 5 Max AVB lock state, boot_control conformance, rollback-index backend — UNVERIFIED [android-avb-rollback §12; android15-virtual-ab §11].
- Default slot-retry-count (commonly 3, sometimes 7; board-dependent) — UNVERIFIED [android-avb-rollback §12].

12. Sources

- [android-update-engine-api.md](#)
- [android-avb-rollback.md](#)
- [android15-virtual-ab.md](#)
- [adr-0001-wrapped-engine.md](#), [adr-0002-supply-chain-trust.md](#)
- [submodule_reuse_map.md](#), [2026-06-07-helix-ota-design.md](#)